

# Full Scale — Hack the North 2026 Build Doc

2026-09-17 · @Someone

## The pitch

**One sentence:** Scan your room and anything you want to put in it, then check at true measured scale whether it actually fits, before you buy it or move it.

## Positioning

Every existing tool solves the *retailer's* problem — place **our** products in your room. IKEA Place, Wayfair, Houzz, Roomvo. Scanning tools like Polycam solve capture but have no room context and no fit logic.

Nobody solves the **resident's** problem, which is the inverse. Most of what goes in a room isn't purchasable right now: the desk from your parents' basement, the Marketplace couch, the bookshelf you already own. And the moment the problem becomes acute is moving day — *will my stuff fit in the new place* — a universal, recurring, entirely unserved need.

So the product is **a measured room plus a library of measured objects**, some from stores and some from the real world. The catalog is an input, not the point.

## The differentiator

Not visualization. **Fit certainty.** Every other tool shows you a picture that is dimensionally a lie. Bulky-furniture returns cost retailers more than the item's margin because reverse logistics on a couch is brutal, so the commercial framing is returns reduction with a measured guarantee — not design inspiration.

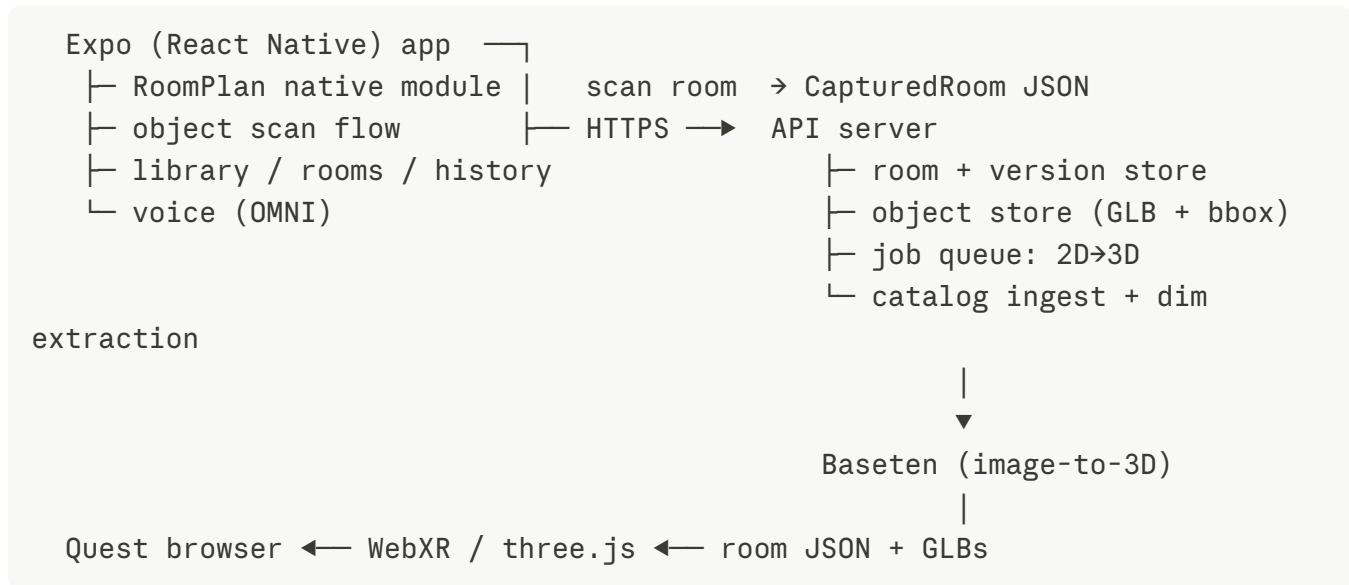
## The line to lead with

Everything that shows you furniture in your room shows you a picture that's dimensionally a lie. This measured your room, measured the object, and it'll tell you when something doesn't fit. And it works on things that aren't for sale anywhere — the desk in your parents' basement, the couch on Marketplace, the stuff you already own.

Then hand them the phone.

## Core tech and architecture

### The shape



### Client — Expo, and why that now makes sense

The earlier advice was to skip Expo because RoomPlan is native-only. **That changes once the phone is the product rather than a scanner.** A library of rooms, a library of scanned objects, version history, sharing — that's a real app worth building in React Native, and the native surface shrinks to one bounded thing.

**The native module.** Expo Modules API, written in Swift. It exposes exactly two things:

1. `RoomCaptureView` as a native view component
2. a callback returning the serialised `CapturedRoom` as JSON

Everything else — list screens, object gallery, history scrubber, settings, handoff — is ordinary RN.

### Setup cost, be aware of it:

- `expo-dev-client` + `npx expo prebuild` + a local Xcode build. No Expo Go.
- Config plugin for `NSCameraUsageDescription`.
- LiDAR device required: iPhone 12 Pro or later, iOS 16+.
- A free Apple developer account works (7-day provisioning profile).



**Time-box the native module to 4 hours.** If it isn't returning JSON by then, fall back to a pure Swift app and drop the Expo track. Do not let this consume Saturday.

**Why this is a strong Expo entry specifically:** most Expo submissions are pure JS. A custom native module wrapping an Apple framework demonstrates the hard part of Expo — escaping the JS sandbox cleanly — and very few teams will do it.

## Transport as JSON, not USDZ

The most important architectural call in this doc. The obvious path is RoomPlan → USDZ → GLB → three.js, and that conversion chain is where the weekend dies: coordinate conventions, unit scaling, material loss, headless Blender in a container at 3am.

Instead, serialise the `CapturedRoom` structure directly — wall transforms and dimensions, opening rectangles, object categories and boxes — as a few kilobytes of JSON, and rebuild geometry in three.js from primitives. Walls are boxes, openings are gaps, furniture is a proxy per category.

Faster to transmit, trivially debuggable, needs no conversion tooling, and produces a *cleaner* room than a photogrammetric mesh would. You lose photorealism you were never going to get.

Segmentation is handed to you by RoomPlan — walls, doors, windows and openings as parametric surfaces, plus objects with category labels, transforms, bounding boxes and confidence values. You are not implementing it. Object removal is `object.visible = false` on something the API already labelled.

## Headset — WebXR, three.js + `@react-three/xr`

No build step, hot reload from a laptop, and the same scene renders in a desktop browser for the casting monitor. Unity would cost five hours in build-and-deploy cycles you don't have.

## Data model

```
Room      { id, capturedRoom, createdAt, versions[] }
Version   { id, parentId, label, createdAt, placements[], materials }
Placement { objectId, transform, flags[] } // flags: blocks_door,
clearance_fail...
Object    { id, source: scan|catalog, glbUrl, bboxMeters,
            confidence, price?, productUrl?, merchant? }
```

Every object, from either ingestion path, converges on { glb, bboxMeters, confidence, source }. The fit engine only ever reads bboxMeters.

## Version history — nearly free, and worth doing

Because a room layout **is** JSON, version control costs almost nothing:

- Content-hash each layout, store a linear history with parent pointers
- Diff two versions → added / removed / moved objects
- Timeline scrubber in the app; **also expose it in the headset**

**Demo value is disproportionate.** Scrubbing a timeline and watching the room reorganise around you is a strong visual beat for maybe 3 hours of work, and it makes the app feel like a product rather than a demo. It also gives the Expo judges something to grade that isn't the headset.

## Ingestion A — scan anything

This is the wedge. Everything else in the product exists because this path does.

**Step 1 — LiDAR box, instant.** Point the phone at an object. The true bounding box comes back in well under a second. This matters enormously for *perceived* latency: a measured box with real numbers on it can render immediately while the mesh is still generating.

**Step 2 — image-to-3D, hosted.** Frames go to a hosted model.

| Model                     | Speed                                                         | Use                              |
|---------------------------|---------------------------------------------------------------|----------------------------------|
| Stable Fast 3D            | sub-second on an A100, ~6GB VRAM, MIT, UV unwrap + PBR params | the live demo path               |
| TRELLIS 2 / Hunyuan3D Pro | slower, prettier                                              | async upgrade, pre-baked catalog |

Multi-view input beats single-image if you have it — a sweep gives you four good frames for free, and it measurably reduces hallucinated backsides.

**Step 3 — the binding, which is the whole technical thesis.** Generated meshes come out at *normalised* scale, roughly `[-1, 1]` per axis. This is a recognised open problem, not a quirk: the Any6D authors state plainly there is no reliable existing solution for single-view metric-scale reconstruction from RGB-D across diverse objects, which is why every consumer image-to-3D tool ships dimensionally meaningless output.

You rescale the generated bounding box to the LiDAR measurement. **Ten lines of code**, and it is the entire reason the AR view is true rather than decorative.

Say this to a technical judge: *the generative model gives us shape, the depth sensor gives us size, and we bind them. That's why this is actually the right size on your floor and not just pretty.*

**Known failure set.** Single-image generation degrades badly on transparent, reflective, thin, and very dark objects. A MacBook is a good demo choice (matte, rectangular, solid). A water bottle is the adversarial case. **Test twenty random objects before Saturday** so you know the boundary, and surface a confidence signal rather than being caught.

## Ingestion B — Shopify products

More involved than path A. This is where the agent story lives.

### Catalog discovery

You **cannot** query "all of Shopify" — the Storefront API is per-store and needs a token per merchant. The practical route:

- Most Shopify storefronts expose their catalog publicly at `/products.json` and `/collections/<handle>/products.json`, no auth.

- Pick **15–25 real furniture merchants**, pull catalogs, index locally.
- ⚠️ Some merchants disable this. **Verify your chosen stores tonight** rather than assuming.

## The messy part: dimension extraction

Shopify's product schema has `weight` but **no standard dimensions field**. Dimensions live wherever the merchant put them:

- custom metafields, inconsistent per merchant
- free text inside `body_html`, in mixed units
- variant titles like `60" x 30"`
- spec-sheet images with no text at all
- nowhere

## Pipeline

1. **Regex first pass.** Numeric patterns adjacent to width/depth/height/W/D/H tokens. Catches ~60% for almost nothing.
2. **LLM pass** over remaining `body_html`, constrained output schema.
3. **VLM pass** on spec-sheet images where text fails entirely.
4. **Validation — this is what Rox actually rewards.**
  - Unit sanity (a sofa is not 8cm wide)
  - Category priors (a dining chair is 40–50cm)
  - Cross-check claimed width:height against the product photo's aspect ratio — a genuine multi-source resolution step, and a good thing to say out loud
5. **Confidence score.** Low confidence surfaces in the UI as **"unverified fit"** rather than being silently guessed. Honest handling of ambiguity is literally in Rox's rubric and is the difference between an agent and a scraper.

## Mesh generation

Primary product image → background removal → image-to-3D → rescale to extracted dimensions.

**Pre-bake 60–100 products offline during the hackathon.** Cache the GLBs. Do exactly **one** live generation on stage, for something the judge names, with the voice agent narrating over the wait.

## Fit engine — the differentiator

**This is 70% of what should come out of your mouth in the pitch.** It is the only part of this project that is unclaimed at this hackathon.

Past Hack the North winners cover the visualisation half thoroughly — SpectraSphere (photos → explorable 3D spaces), Shop The North (VR shopping), Flashback (VR 3D exhibits), ShapeShift, Aircraft Studio. **Nobody has won here by proving the generated thing is dimensionally true.**

### What it checks

- **Clearance corridors.** Walking paths between placed objects, against a configurable minimum (90cm is a good default; it is also the AODA/ADA neighbourhood).
- **Door swing arcs.** RoomPlan gives you door position and width. Sweep the arc, intersect with placed geometry.
- **Window occlusion.** Does this block the light source.
- **Wall adjacency.** Is it actually against the wall or floating 4cm off it.

### The demo beat nobody else can do

The solver returns **infeasible**, and the system says so:

*There is no arrangement of these three pieces that keeps a 90cm walkway to the door.  
Drop the ottoman, or go 20cm narrower on the couch.*

A system that can be wrong and knows it. Everyone else's demo is vibes.

### Implementation note

Do **not** let an LLM emit furniture coordinates. The 2025 Shopify track winner (Tarazoo) paired camera-based checkout with a real MINLP procurement optimiser — those judges rewarded actual optimisation under a legible demo.

The LLM's job is to translate "*cozy reading corner*" into an objective function and constraints. A solver places things. Discretise the floor to a grid, run `scipy.optimize` or OR-Tools. **~4 hours**, and it is the difference between this project and the other room-design projects on the floor.

## AI redesign — scope it hard

**Verdict: do it, but not the way it's usually imagined.**

Generating room *geometry* is off the table. It would look worse than your scan and take longer than you have. Never attempt generative geometry this weekend.

But the "reality stone" effect decomposes into two cheap operations on things you already have:

1. **Material swap.** Wall colour, floor material, trim. Your walls are parametric surfaces with assigned materials, so this is an *assignment*, not a *generation*. Instant, and it carries most of the visual payoff.
2. **Bulk object substitution.** Every furniture proxy swaps to a different set in one animated beat. Also cheap. Visually it reads as total transformation.

Run both simultaneously on one voice command and you get the full *the room becomes something else* moment for a few hours of work, with zero generative risk.

## The best version of the command

Budget-constrained:

**"Redo this under \$1,200."**

Items visibly swap around you while a running total drops. That is a Shopify-track demo beat and it is spectacular in a headset — the agent re-solves the whole room with cheaper substitutes while you stand inside it.

## Furniture-only vs whole-room

Whole-room, but only in the restricted sense above. Furniture-only substitution is less impressive for the same effort, since materials are nearly free once the geometry is parametric.

## Voice

**Why it earns its place:** you have no keyboard in a headset, and controller text entry is miserable. Voice is not a garnish here, it is the only sane input for *“put a reading chair in that corner”* or *“make the walls sage green.”*

### Build it on the phone first

Native **Speech** framework, on-device, zero latency, completely reliable. The phone is already in the demo and already in someone’s hand.

**In-headset voice is a stretch goal.** Quest browser speech support is inconsistent. If you go that way, use push-to-talk on the controller streaming audio to a server — do not trust the Web Speech API to be there.

### What it buys beyond usability

- The **speech modality** for Huawei OMNI Live (vision + speech + language on an edge device is their hard requirement)
- A natural **ElevenLabs** integration for responses
- Cover for generation latency — a narrating agent turns 15 seconds of dead air into 15 seconds of progress

## Cheap high-value adds

### Sun simulation — the second measurement claim

Fit answers *does it physically go there*. Sun answers *should it go there*. Together: **we tell you where things go, with reasons you can check.**

**Do not hardcode it.** Solar position from latitude, longitude and timestamp is a closed-form calculation (NOAA solar position algorithm, ~40 lines, no network, no API key). Faking it costs credibility for zero savings. The only thing worth approximating is sky and ambient colour — use a lookup table keyed on sun elevation rather than attempting atmospheric scattering.

## ⚠ The gotcha that would silently make this fake

RoomPlan gives window rectangles in **room coordinates**. The room has no absolute orientation, so by default you have no idea which way it faces — and the sun would come from an arbitrary direction while looking completely plausible.

**The fix is one enum value:** set `ARConfiguration.worldAlignment = .gravityAndHeading` on the capture session. The ARKit world frame is then aligned to true north, and window normals become real compass bearings. Store the bearing with the room.

Without this, the whole feature is a lie — which is a bad thing to ship in a project whose entire pitch is *other tools are lying to you*.

### Also watch

- **Timezone and DST.** Compute in UTC, convert once for display. Classic silent bug.
- **Shadows.** One directional light + shadow map. Do not attempt global illumination.
- **Light through windows.** Cast the directional light through the window opening and let the shadow map produce the floor patch. Volumetric shafts are a stretch goal, not a requirement.

### The upgrade worth taking: daylight heatmap

Run the sun loop at 15-minute intervals across a whole day, accumulate direct-light exposure per floor cell, render as a colour overlay.

*This corner gets 3 hours of direct light per day in November. That's why your plants keep dying.*

Cheap (it's the same loop in a for-loop), instantly legible to anyone glancing at the screen, and it converts a pretty effect into **a number** — which is the thesis of the whole project.

**Cost:** ~2h base, ~3h with the heatmap.

### Positioning caution

Keep this as the **second demo beat, not a co-pitch**. "Place your lamps better" is a weaker proposition than fit certainty, and leading with it dilutes the one thing here that's unclaimed. It's a great *moment*; it is not the product.

## Shareable output

Generate a link or QR of the final room that opens in any browser. Judges like being able to keep something. Near-free once the scene is already JSON.

## Phone-AR bookend

Hand the phone back so they can hold the placed object up against the real space in AR. Nice closer, first thing to cut if you're behind.

## Demo choreography

### Station setup

- **Monitor casting the Quest view.** A teammate wears the headset. This is non-negotiable — without it, four bystanders see nothing during your best moment.
- **Phone on the table**, unlocked, camera session already warm.
- **A tape measure on the table.** Cheapest credibility you will ever buy. It converts every claim into something the judge can check themselves.
- **A travel router or laptop hotspot**, all devices on it. ~\$40. Eliminates your single largest latency risk, because conference Wi-Fi at peak judging will not carry phone → server → Quest.

### The 60-second run

1. Room is already loaded, teammate already walking it on the monitor.
2. Judge picks up the phone, scans their MacBook.
3. **Bounding box snaps on instantly with real numbers.** *"It's already measured — 30.4 by 21.2 centimetres. The mesh is generating."*
4. 10–15s of narration while it generates. **This is where the thesis line goes:** *every other tool shows you a picture that's dimensionally a lie.*
5. MacBook drops into the room at true scale on the big screen. Teammate walks around it.
6. Voice: *"put a reading chair in that corner."* A real product arrives. **Fit engine fires:** *"that blocks the door swing by 11cm"*, arc drawn in red.
7. Voice: *"show me 4pm in November."* Sun angle shifts, light leaves the corner.
8. *"Want to try it?"* — headset goes to whoever lingers.

## Judge parallelisation

Judges arrive in clumps. **Formalise this.** Whoever is holding the phone is the participant; everyone else watches the monitor. One judge scans while the others watch the previous result. Generation latency becomes fill time rather than dead air.

The headset is an **upsell for judges who linger**, never a bottleneck in the main path. This fixes both the latency and the throughput problem at once.

## Build order and scope triage

### Hours 0-4 — before any features

These three are not the fun part. They are where this project fails if it fails.

1. **Expo native module returns `CapturedRoom` JSON.** Time-boxed to 4h. Miss it → pure Swift, drop the Expo track.
2. **RoomPlan JSON → three.js room in the Quest browser at correct scale.** Hardcoded scan is fine.
3. **Phone → server → Quest push of a hardcoded GLB.** Prove the transport.
4. **Baseten image-to-3D returning a mesh**, timed end-to-end *on the venue network*.

If these aren't working by hour four, you need to know **then** — not at hour thirty.

### Core (non-negotiable)

- Object removal (free — RoomPlan already labelled them)
- Scan-an-object → generated mesh → **true-scale binding**
- Move and rotate
- **Fit engine with obstruction warnings**

### High value, cheap

- Sun simulation (~2h)
- Material + furniture restyle
- Voice on phone

Cut first, in this order

- 1. Agentic webscraping beyond the dimension extractor
- 2. Shopify checkout
- 3. In-headset voice
- 4. Multi-room
- 5. Anything generative beyond materials

Two things to verify *tonight*

- **Who owns Swift.** RoomPlan and LiDAR are native-only. Without a comfortable iOS person the measurement claim — the entire differentiator — falls back to estimates, and the project becomes a nice-looking wrapper.
- **Do your chosen merchants expose `/products.json`.** Ten minutes, and it determines whether ingestion path B exists at all.

Sponsor tracks — definitive

**Selections lock Saturday ~2:00 PM**, roughly 20 hours in. You will be choosing before you know what's finished, so the list below is split by what each track is *contingent on*.

 **There is no AR/VR track in 2026.** Ubisoft ran one in 2024, Snap in 2025. Nothing this year. Every hour on the Quest earns zero sponsor points. It's still your demo — but it is spend, not investment. Budget accordingly.

Tier 1 — build for these

| Track                                | What specifically earns it                                                                                                                                                                                  | Contingent on                   |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>Expo</b> — Best Mobile Experience | The whole product is an Expo app: rooms, object library, version history, scan flows. A <b>custom native module wrapping RoomPlan</b> is the differentiator — most Expo entries never leave the JS sandbox. | Native module working by hour 4 |

| Track                   | What specifically earns it                                                                                                                                                                                                                                         | Contingent on                                  |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>Baseten</b>          | Hosts image-to-3D. Cannot run on the phone, so hosted inference is load-bearing, not garnish. <b>Two Baseten people are on the judging panel</b> — the best attention signal available this year. Run two latency tiers: Stable Fast 3D live, TRELLIS 2 async.     | Nothing. This is the safest track on the list. |
| <b>Huawei OMNI Live</b> | Vision (room + object capture), speech (voice commands), language (intent → scene edits) — all on a phone. Their three-modality requirement is a hard filter that washes out most entrants, so the field should be thin. Cloud API access is explicitly permitted. | Voice loop shipping, which is cheap            |

## Tier 2 — select only if the work actually lands

| Track                 | What specifically earns it                                                                                                                        | Contingent on                                                            |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Shopify</b>        | Real products with real dimensions placed at true scale; the budget resolve.                                                                      | Catalog ingestion shipping — <b>the most cuttable piece in the build</b> |
| <b>Rox</b><br>(\$10K) | The dimension extraction pipeline: conflicting sources, unit validation, aspect-ratio cross-check, confidence scoring, "unverified fit" flagging. | Same. No extractor, no Rox.                                              |

## Tier 3 — free

**ElevenLabs** or **Gemini**, whichever half of the voice loop OMNI isn't covering. **Sentry** if you're instrumenting anyway.

## The call

**Expo is the primary.** It's the only track where the thing you're most committed to building — a polished iPhone app with real native integration — *is* the submission. Baseten and OMNI are near-certain secondaries. Shopify and Rox are the prize-heavy pair, and both hang on the same at-risk workstream, so treat them as one bet rather than two.

**Four to six total.** Twelve shallow ones reads as desperate and dilutes every pitch. Sponsors with specific rubrics can tell token integration from real.

## Risks and kill criteria

Structural (won't go away with better execution)

**Originality.** "AI makes 3D, you look at it in a headset" is a well-worn lane here with multiple entrants every year, and returning judges have seen versions three years running.

**Countermeasure: lead with measurement, not visualisation.** If the fit engine is the centre of the pitch, this weakens a lot. If it's an add-on built at hour 30, it doesn't.

**No AR/VR track.** Covered above. Accept it as the cost of the demo.

## Mitigable

| Risk                                                                | Mitigation                                                             |
|---------------------------------------------------------------------|------------------------------------------------------------------------|
| 10–20s scan-to-headset latency                                      | Instant LiDAR box + narration + judge parallelisation + own router     |
| Headset handoff eats demo time                                      | Casting monitor; headset is an upsell, not the main path               |
| Judge doesn't supply the room                                       | They supply the object. Lean on that hard.                             |
| Mesh quality on hard objects                                        | Test 20 objects Friday. Surface confidence. Steer toward matte solids. |
| USDZ→GLB conversion hell                                            | <b>Avoided by design</b> — JSON transport, rebuild from primitives     |
| Pipeline fragility (most integration points of anything considered) | The hours 0–4 checklist exists precisely for this                      |

## Kill criteria

- **Hour 4:** if RoomPlan → Quest at correct scale isn't working, stop and reconsider scope. Do not push through on hope.
- **Hour 20 (track lock):** if the fit engine doesn't exist yet, you have a visualisation project, not this project. Select tracks accordingly.
- **Saturday afternoon:** rehearse on the venue network *when the floor is full*, not at 3am with the AP to yourself. This catches more demo failures than anything else you can do.